

INSTAGRADER

by

Nathan Fredericks

Jodrey School of Computer Science

COMP 4983

Supervisor: Dr. Sazia Mahfuz

Acadia University

April 2026

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
1.3	Goals and Methods	2
2	Background	3
2.1	Automated Essay Scoring	3
2.1.1	History of Automated Essay Scoring	3
2.1.2	Trait-Based vs. Holistic Scoring	4
2.2	Large Language Models and Fine-Tuning	4
2.3	Training Data	5
2.3.1	ASAP Dataset	5
2.3.2	ASAP++ Dataset	6
2.4	Related Work	6
2.5	Privacy and Ethics	7
3	Implementation	9
3.1	System Architecture	9
3.1.1	Technology Stack	10
3.2	Hardware and Software Requirements	11
3.3	Dataset Preparation	11
3.4	Model Training	12
3.5	Backend	13

3.6	Frontend	14
3.7	API Design	15
3.8	Grading Workflow	16
3.9	Testing and CI/CD	17
4	Conclusion	19
4.1	Conclusion	19
4.2	Future Scope	20
4.3	Reflections	21
A	Installation and Usage Guide	23
A.1	Prerequisites	23
A.2	Installation	24
A.3	Usage	25
A.3.1	Creating a Rubric	25
A.3.2	Creating an Assignment	25
A.3.3	Uploading Essays	25
A.3.4	Reviewing Results	25
A.3.5	Completing an Assignment	26
	Bibliography	27

List of Tables

3.1	Assignment states	13
3.2	API endpoint groups and their operations.	15
4.1	Technology pivots from proposal to final implementation.	21

List of Figures

3.1	System architecture and data flow for InstaGrader.	10
-----	--	----

Chapter 1

Introduction

1.1 Motivation

Every year, teachers are overwhelmed by grading. One essay can take 5 to 15 minutes to read, grade, and provide feedback on. A teacher with 30 or more students per class, across multiple classes, can spend dozens of hours per assignment grading alone. This repetitive workload contributes to teacher burnout and high turnover [37, 28].

The time it takes to receive feedback on an assignment also affects students. When grading takes weeks, students get feedback a long time after they’ve moved on from the assignment. Late feedback isn’t as effective as timely feedback for improving student writing [23]. Students get the most benefit when they can revise while their assignment is still fresh in their minds.

Existing grading tools, like CoGrader [13] and EssayGrader [17], use third-party LLM APIs to score essays against rubrics. They all require sending student essays to third-party servers, which may violate student data privacy under Canadian privacy legislation.

1.2 Problem Statement

There are no widely accessible tools that give teachers AI-assisted essay grading while keeping them in full control of both the review process and their students’ data. The tools available today require teachers to trust third-party API providers with their students’ writing, which

Chapter 1. Introduction

may not comply with their institution’s or provincial privacy requirements. Privacy aside, existing tools also suffer from poor-quality feedback. Most automated essay scoring systems (AESs) produce a single holistic score. A holistic score tells a student how they did overall, but not what to specifically improve. Trait-level feedback, which breaks grading down into individual rubric criteria such as grammar, organization, and clarity, is much more helpful to students [30].

1.3 Goals and Methods

The goal is to fine-tune a large language model on the ASAP and ASAP++ datasets [22, 29] so it can score essays for each rubric criterion. The model should be able to create a numeric score and written feedback for each criterion. This will give teachers a starting point for grading.

The model itself is not useful to teachers. They need a user interface to interact with. The second goal is to build a full-stack web application around the model that allows teachers to create and edit rubrics and assignments, upload student essays in bulk, and review AI-generated scores. The application should be simple, familiar, and easy to use.

I made several technical decisions that support these goals. The model is based on `gpt-oss-20b`, an open-source model released by OpenAI in 2024. The model is deployed on AWS Bedrock [1], so it can be scaled serverlessly without requiring a powerful, expensive server. Grading happens in the background using Celery [38] and Redis [35], and the user interface keeps the user up to date on the status of each grading job. On the backend, every essay must be reviewed by the teacher before marking can be completed, keeping the decision-making power in the hands of teachers.

To try to keep the codebase reliable while working on the project, I wrote automated tests and a GitHub Actions workflow. The backend tests cover every single endpoint and as many edge cases as I could think of. The GitHub Actions workflows run tests, lint the code, and verify that the frontend’s API types are in sync with the backend’s types.

Chapter 2

Background

2.1 Automated Essay Scoring

2.1.1 History of Automated Essay Scoring

Automating essay scoring has a long history [14]. The first attempt was by Ellis Page in 1966 [33], who developed the Project Essay Grade (PEG). It used features of the essay, such as word count, sentence length, and punctuation frequency, to determine the quality of the writing. The project was quite rudimentary, but it showed that numerical patterns in text could predict essay scores with some accuracy.

Over the next 30 years, AES became more advanced as research into natural language processing increased. In 1988, ETS created e-rater [11]. E-rater used natural language processing features, such as essay structure, vocabulary, and the number of grammatical issues, to determine a score. It was used for grading hundreds of thousands of GRE and TOEFL assessments [7]. A few years later, Landauer et al. developed the Intelligent Essay Assessor (IEA) [19]. This used Latent Semantic Analysis [27] to see how closely a student’s writing matched reference writing.

Since then, advances in machine learning have moved AES research towards neural network approaches [36]. LSTMs and transformer models outperform older feature-based models on benchmark datasets. When the Kaggle ASAP competition was held in 2012, it attracted 154 teams and produced the first large-scale public benchmark for AES [22]. More

Chapter 2. Background

recently, large language models (LLMs) have enabled trait-level scoring of essays with written feedback [30]. Legacy systems could not provide this kind of detailed scoring.

2.1.2 Trait-Based vs. Holistic Scoring

There are two approaches to scoring: holistic and trait-based. Holistic scoring assigns a single overall score to each essay. This summarizes its quality into one number. It’s faster for graders to review and edit, but it doesn’t give students any information about how they did or what they need to improve.

Trait-based scoring works differently. It evaluates an essay across multiple rubric criteria separately (e.g., quality of thesis, evidence use, organization, word choice, fluency, and conventions). Each criterion gets its own score and its own feedback. This makes trait-level scoring better for students because they can see exactly where they need to focus and improve, with specific examples from their paper.

InstaGrader uses trait-based scoring. The system scores essays using the teacher-provided rubrics, producing scores and feedback for each criterion. The system’s LLM is trained on a combination of the ASAP [22] and ASAP++ [29] datasets. The ASAP++ dataset extends the base ASAP dataset by providing annotations for each trait. Prompts 1 and 2 are scored across five criteria (Content, Organization, Word Choice, Sentence Fluency, and Conventions), while Prompts 3 through 6 use four different criteria (Content, Prompt Adherence, Language, and Narrativity). By combining a fine-tuned LLM with structured rubric data provided by teachers, InstaGrader can provide trait-level feedback that holistic scoring can’t.

2.2 Large Language Models and Fine-Tuning

Large language models are neural networks trained on large amounts of text data, which lets them generate, summarize, and analyze textual content. Models like OpenAI’s GPT-5.4 and Anthropic’s Claude Opus 4.6 [5] are pre-trained on billions of tokens from books, websites, and public code repositories. This broad training gives them a general understanding of human language, but they are not specialized for any one task out of the box.

2.3. Training Data

Fine-tuning is the process of adapting a pre-trained model to a specific task. It means further training the model on a smaller, more specific dataset. For essay scoring and feedback, the InstaGrader model uses a dataset of thousands of essays paired with rubrics and human-graded scores. Through this, the model learns to map the essay’s content to different rubric criteria and generate scores and feedback similar to how human graders would grade the same work.

LLMs can score essays through prompting, without any fine-tuning, but fine-tuning comes with a handful of advantages: fine-tuned models produce more consistent scores across similar essays, follow rubric definitions better, and cost less because smaller fine-tuned models can match the performance of larger prompted ones [24]. InstaGrader fine-tunes `gpt-oss-20b` using AWS SageMaker AI [2]. The model gets the essay text, a structured rubric, and the writing prompt as input and outputs a JSON object with scores and feedback for each criterion.

2.3 Training Data

2.3.1 ASAP Dataset

The Automated Student Assessment Prize (ASAP) was released in 2012 as a part of a Kaggle competition [22]. It has over 12,000 essays written by students who were given one of eight writing prompts. The essays were written by students in grades 7 to 10 and have narrative and argumentative writing styles. Each essay was graded by multiple human raters, and the dataset includes scores on prompt-specific scales.

Because of the dataset’s size and quality, ASAP quickly became the benchmark for AES research. The competition in 2012 brought in 154 teams. One downside of ASAP is that it provides only a single holistic score for each essay. There are no breakdowns by rubric criterion, which means models trained on ASAP alone cannot produce the trait-level feedback that is most useful to students. InstaGrader uses prompts 1 through 6 from ASAP. Prompts 7 and 8 are excluded because they use a different scoring format and are not included in ASAP++.

Chapter 2. Background

2.3.2 ASAP++ Dataset

To improve on the ASAP dataset, Mathias and Bhattacharyya [29] released the ASAP++ dataset. ASAP++ extends the original ASAP dataset by adding trait-based scores. The dataset is provided as six CSV files, one for each prompt. Prompts 1 and 2 are scored across five criteria: Content, Organization, Word Choice, Sentence Fluency, and Conventions. Prompts 3 through 6 use a different rubric with four criteria: Content, Prompt Adherence, Language, and Narrativity.

ASAP++ enables fine-tuning the model used by InstaGrader. Without having scores for each criterion, the model can only learn to produce holistic scores. With the ASAP++ dataset, the model can learn the relationship between essay content and individual rubric scores. InstaGrader merges the original ASAP essay text with ASAP++ trait-level scores to create training samples that pair each essay with its prompt, rubric, and individual criterion scores. The combined dataset comprises 10,686 essays across 6 training prompts.

2.4 Related Work

There are several commercial tools that already use LLMs to help teachers grade essays. CoGrader [13] is a web application that uses external APIs (such as OpenAI’s) to grade student essays against teacher-provided rubrics. It lets teachers review AI-generated scores before exporting them. EssayGrader [17] also offers a similar service, but it offers fewer options for customizing the rubric. Both of these applications require that student essays be sent to third-party API providers for processing.

By relying on an external API, CoGrader and EssayGrader are entrusting a third party with students’ personal data, which raises privacy concerns. In Canada, student data is protected by federal and provincial privacy legislation. PIPEDA [21] governs how private-sector companies collect and use personal information. Individual provinces, like Nova Scotia, have their own public-sector privacy acts that apply to schools. Teachers and school boards might be hesitant to adopt tools that transmit students’ work to third-party servers outside of their control.

In academic research, AES has produced some very advanced scoring models, but very

2.5. Privacy and Ethics

few of them have been deployed as usable tools for teachers. Most research systems focus on the model’s accuracy on benchmark datasets (such as ASAP) and publish their results in papers without building a full application around them. There is a gap between these research models and what is available for teachers to use in practice.

InstaGrader aims to close that gap. It’s different from existing tools in three main ways: (1) it uses a custom fine-tuned model on infrastructure controlled by me; (2) it includes a full rubric designer where instructors can build custom rubrics with drag-and-drop criteria and different scoring levels, and (3) it keeps control in the hands of teachers with a grading workflow that focuses on human assessment. Every AI-generated score must be reviewed by the teacher, and teachers must write their own feedback before each score can be finalized.

2.5 Privacy and Ethics

Student essays often contain personal information protected under Canadian privacy laws. At the federal level, PIPEDA [21] governs how private-sector companies can collect, use, and disclose personal information. For provincial public bodies, such as schools, provincial privacy legislation applies. In Nova Scotia, the Freedom of Information and Protection of Privacy Act (FOIPOP) [34] requires public bodies to protect personal information they hold and prevents or limits its disclosure to third parties. Essay content can include personal details, opinions, and circumstances that students might not want anyone but their teacher to see.

InstaGrader tries to address these concerns by storing student data on its own infrastructure. The fine-tuned model runs on AWS Bedrock [1] within my AWS account, and is hosted in the `ca-central-1` region in Montreal, Canada. Essays are never sent to any third party, like OpenAI or Google. This means that student data can be processed in a way that I can control and fully audit, and it stays within Canadian borders.

Privacy is one concern, but ethical considerations matter as well. A key question is how much control AI should have over student grades. InstaGrader takes the approach that AI-generated scores are *not* the final grade. Every score must be reviewed by the teacher before it is given to the student, and all feedback must be written by the teacher. This encourages the teacher to think critically about the grade that they are assigning. The teacher can

Chapter 2. Background

accept or override any AI-generated output. I designed it this way with the intention that AI grading should support teachers, not replace them.

There is also risk of bias in the training data. The ASAP and ASAP++ datasets have essays from students in grades 7 to 12, who are responding to different prompts. The model might be less accurate on essays from different grade levels, subjects, or writing styles that weren't represented in the training data. I try to reduce this risk by making the AI's reasoning visible to the teacher so that they can see each score and judge if the AI's score makes sense for that specific criterion.

Chapter 3

Implementation

3.1 System Architecture

InstaGrader is built in three parts: a frontend built with React [31], a backend built with Django [15], and an AI model hosted on AWS Bedrock [1]. The frontend communicates with the backend using a RESTful API. The REST API schema is defined in an OpenAPI 3.0 specification. When a teacher submits a stack of essays for grading, the backend assigns the work to a pool of Celery [38] workers, each running in a separate process and calling the fine-tuned version of the model on AWS Bedrock.

Several services connect these parts. Redis [35] serves as the message broker and result backend for Celery. This allows Celery to queue grading tasks and store their results until the backend polls for them. The database stores all application data, including users, rubrics, assignments, essays, and grading results. Authentication is handled using JSON Web Tokens. I store the token in a cookie on the frontend. The token lasts for 60 minutes and can be refreshed for up to 7 days.

Figure 3.1 shows how these parts connect. A request starts in the browser, passes through the Next.js server for authentication and rendering, and then reaches the Django API. When the API receives a grading request, it hands the task off to Celery. Redis sits between Django and Celery as the message broker, holding tasks in a queue until a worker picks them up. The Celery worker then calls the fine-tuned model on AWS Bedrock and writes the result back to Redis, where the backend can retrieve it.

Chapter 3. Implementation

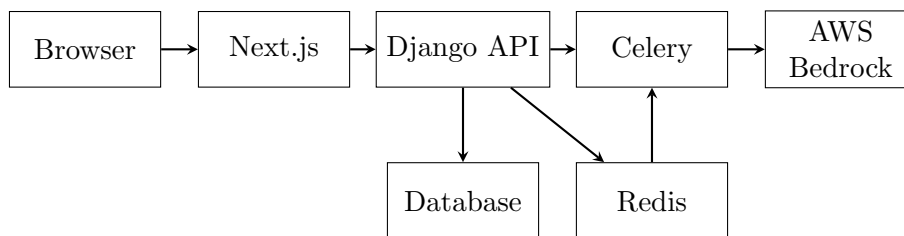


Figure 3.1: System architecture and data flow for InstaGrader.

3.1.1 Technology Stack

The frontend is built with Next.js [40] and React [31]. I chose Next.js because it supports server-side rendering, and I’m familiar with it from previous projects. Since it’s server-side rendered, I can verify user authentication and fetch initial data on the server before the page reaches the browser. The user interface is built with shadcn/ui [8], a component library. It’s styled with Tailwind CSS [41].

On the backend, InstaGrader uses Django [15] with Django REST Framework [12]. I originally planned to use FastAPI in the proposal, but I changed my mind and used Django instead because it has built-in authentication, an admin interface, and an ORM I was already familiar with from my previous co-op. Django REST Framework adds serialization, viewsets, and permissions to Django. I added the drf-spectacular [25] package to generate the OpenAPI schema, and a GitHub Actions workflow to keep the schema in sync with the frontend. At my last co-op, we used both Django REST Framework and drf-spectacular, so I followed the same approach.

Since grading each individual essay can take time, grading needs to happen in the background so users can do other things on the site. InstaGrader uses Celery [38] with Redis [35] as its message broker to run grading tasks without blocking the user interface or forcing users to wait. The fine-tuned model is hosted on AWS Bedrock [1], a service from Amazon Web Services that handles hosting, scaling, and inference. AWS Bedrock is serverless, which makes it affordable for a student like me who does not have the hardware required to run model inference locally. Lastly, I use the MarkItDown [32] package from Microsoft to extract text from the uploaded PDF, DOCX, and TXT files before sending them to the model.

3.2 Hardware and Software Requirements

InstaGrader has very few system requirements and has been tested on a server with as little as 2 vCPU and 2 GB of RAM. The project’s backend requires Python 3.13+. Python 3.14 is not supported because of dependency incompatibilities. The project’s frontend uses the Bun runtime [39]. Redis [35] is used as the message broker for Celery. For AI inference, the application calls a fine-tuned `gpt-oss-20b` model that’s hosted on AWS Bedrock in `ca-central-1`. This requires an AWS account with Bedrock access enabled. Detailed, step-by-step installation instructions are in the Appendix.

3.3 Dataset Preparation

The ASAP dataset [22] is a single file containing 12,979 essays. The ASAP++ dataset [29] comprises six CSV files containing scores for each trait. Only the first six prompts were used; the 7th and 8th were excluded because they used a different scoring format and were not included in ASAP++. After filtering, the working dataset has 10,686 essays.

The two datasets needed to be merged and restructured to train the model. I built two Python scripts to join the essay text from ASAP with the per-criterion scores from ASAP++. I matched the essays by their IDs. I mapped each criterion name (Content, Organization, Word Choice, etc.) and its scores to a numeric ID that corresponds to a structured rubric in JSON. The two base rubrics cover all six of the prompts. There is one rubric for prompts 1 and 2, and another for 3 through 6.

The merged data is converted into the format needed for fine-tuning. The output is an `essays.json` file containing roughly 7,000 essays, each with the essay text, prompt ID, rubric ID, and mapped criterion scores. This is then converted to JSONL format for AWS Bedrock fine-tuning, split into a training set of 8,497 samples and a validation set of 2,126 samples.

The training data also needed written feedback for each criterion, instead of just numeric scores. Since the ASAP++ dataset contains only numeric scores without explanations, I used a batch job with Claude Sonnet 4.5 [4] to generate per-criterion feedback for each essay. I gave the model the essay, its rubric, and the human-assigned scores, and it produced a short

Chapter 3. Implementation

explanation of why each score was assigned. This produced the `essays_with_feedback.json` file that has around 313,000 lines.

The feedback portion of the training dataset was generated artificially. The numeric scores were provided by human raters in the original ASAP++ dataset, while the written feedback was generated by a large language model. This means the fine-tuned `gpt-oss-20b` model is trained to produce feedback like Claude Sonnet 4.5 would, rather than how a human teacher would write feedback. The numeric scores are determined by humans, but the feedback is a synthetic approximation of what a teacher might write. This is a limitation of the training approach I took, and the human-focused review process helped reduce it by requiring teachers to write their own feedback before passing it on to a student.

3.4 Model Training

The grading model is a fine-tuned version of `gpt-oss-20b`, an open-source 20-billion-parameter LLM released by OpenAI in 2024. The fine-tuning was performed on AWS SageMaker AI [2] using supervised fine-tuning (SFT), where the model is trained on input-output pairs of essays and their corresponding scores. The training data has 8,497 examples, and the validation set has 2,126 examples.

The model’s input and output are structured as JSON and designed to work with InstaGrader’s backend. Each example has a prompt structured using OpenAI’s Harmony format, which is similar to XML. The prompt includes the system instructions, the writing prompt, optional reference text, the rubric as a JSON object, and the essay text. The expected output is a JSON object with an array of scores. Each score has a `criteria_id`, a `level_id`, and a feedback string.

The fine-tuning used the following hyperparameters: batch size = 8, learning rate = 0.0001, 5 training epochs and a learning rate warmup ratio of 0.1. During inference, the temperature is set to 0.0 to reduce randomness and prevent essays from being scored differently each run.

3.5 Backend

The backend is organized into four Django apps. The accounts app handles the user registration and authentication. The rubrics app manages creating and editing rubrics, and the assignments app handles essay uploads and the grading workflow. The grading app coordinates essay scoring and stores the results.

Authentication is built upon Django’s built-in User model. The model uses UUID primary keys instead of the default sequential primary key for security, and email-based login instead of usernames, so forgot-password functionality could be added later. The user’s name is stored in a `full_name` field. JSON Web Tokens are stored in HttpOnly cookies to prevent the frontend from accessing them.

The rubric system is the basis of how InstaGrader scores essays. The rubric data model is split into three parts. First, a Rubric object has an ordered list of `RubricCriteria`; from there, each `RubricCriteria` has an ordered list of `CriterionLevels`. This structure maps to the JSON format that’s used by the fine-tuned model. InstaGrader includes three built-in rubric templates that teachers can use as starting points.

Assignments and their essays follow a status pipeline that tracks their progress through the grading workflow. An assignment moves through four states, summarized in Table 3.1.

Table 3.1: Assignment states

State	Description
DRAFT	Created, no grading yet
GRADING	AI processing in progress
REVIEW	All essays graded, awaiting teacher review
COMPLETED	All reviews finalized

Each essay within an assignment has its own status: PENDING, PROCESSING, GRADED, REVIEWED, or FAILED. The essay model stores both the original uploaded file and the extracted raw text.

The grading results are designed to keep teachers as the primary decision-makers. Each essay has a `GradingResult` object that contains `CriterionScore` objects for each rubric

Chapter 3. Implementation

criterion. Each `CriterionScore` stores the automated level and feedback, plus the teacher’s overrides. This lets teachers either accept the AI’s scores or replace them.

Teachers can upload essays in several formats. The system accepts PDF, Word, and text files, as well as ZIP files with multiple essays. The backend rejects unsupported formats and filters out hidden files like macOS’s `.DS_Store` files.

3.6 Frontend

The frontend was built with Next.js [40], the App Router, and React 19 [31]. I organized the app into two groups: `(auth)` for the login and signup pages, and `(app)` for all authenticated routes. I did this because authentication routes use different layouts than authenticated ones, so it was easier to keep them separate.

The authentication is handled entirely on the server. A proxy checks for JSON Web Token cookies on every API request and, if the user is unauthenticated, redirects them to the login page. The `getCurrentUser()` function checks on the user’s session by calling the backend’s `/api/auth/me` endpoint. The login, signup, and logout actions are implemented as Server Actions with React’s `useActionState` hook. This handles form submissions and error states without additional client-side JavaScript.

The app has four main areas. The dashboard, where teachers can see their grading progress and summary statistics, and the assignment pages, where instructors can create assignments, upload essays, view individual assignment grading progress, and review results. The rubric pages let teachers make rubrics from scratch or from a template, with a drag-and-drop designer to arrange criteria and scoring levels. The essay review page is where instructors can see individual AI-graded essays and approve or override their scores.

Several custom components required significant implementation effort. The rubric designer is the most complex component in the whole application. It uses the `dnd-kit` [9] library for drag-and-drop reordering of the criteria and levels, and includes validation and data persistence. The essay-review component displays the criterion scorecards alongside the essay text, with inline feedback and the ability to navigate between essays. The grading progress view polls the backend every 5 seconds during batch grading to show real-time progress. The essay upload drop-zone supports drag-and-drop uploads of multiple files or ZIP archives.

3.7. API Design

The frontend communicates with the backend using typed API calls. The `openapi-fetch` [16] library auto-generated TypeScript types and an API client from the backend's OpenAPI schema. This ensures that the API calls on the frontend match what the backend will return. The app's UI is styled with `shadcn/ui` [8] components and Tailwind CSS [41]. A `SessionWatcher` component checks the access token every 50 minutes and refreshes it to keep the user logged in while they use the app.

3.7 API Design

InstaGrader's backend exposes an API with an OpenAPI schema generated by `drf-spectacular` [25]. The schema is the source of truth for the API between the frontend and the backend. `drf-spectacular` renders a user interface to browse the API schema at `/api/docs`.

The API is organized into four different groups of endpoints. Table 3.2 summarizes each group and the operations it supports.

Group	Operations
Auth	Register, login, refresh, logout, me, profile update, change password
Rubrics	CRUD, structure update, duplicate, reorder, templates list, template instantiation
Assignments	CRUD, essay upload (ZIP or individual), essay list, CSV export, PDF export
Essays/Grading	Essay detail/delete, grading result, score approval/override

Table 3.2: API endpoint groups and their operations.

To keep the frontend and backend in sync, I wrote a GitHub Actions workflow that regenerates the TypeScript types from the OpenAPI schema on every pull request. The `api-types.yml` workflow runs `drf-spectacular` to export the schema, then generates a TypeScript definition file (`schema.d.ts`) using `openapi-typescript` [16]. If the generated types differ from the committed ones, the workflow fails, alerting developers to schema drift before it reaches production.

Chapter 3. Implementation

3.8 Grading Workflow

The grading workflow is the most important part of InstaGrader, and is what distinguishes it from competing platforms. Its goal is to make it easy for teachers to grade essays without taking away their autonomy. The grading workflow follows a linear progression: assignment creation, essay upload, grading, teacher review, and approval.

The process begins when a teacher creates an assignment. They provide a title, a writing prompt, reference material, and the option to select a rubric. From there, they can upload essays either through drag-and-drop or a selection interface. They can upload PDF, Word, and text files, or ZIP archives containing multiple essays.

Once essays are uploaded, the system sends the work to background workers. InstaGrader uses Celery [38] to manage these workers. First, a worker extracts the essay text from the uploaded file using Markdown [32]. Then, every essay is graded in parallel, with up to 10 workers running simultaneously. The worker sends the extracted text, writing prompt, and rubric JSON to the fine-tuned `gpt-oss-20b` model on AWS Bedrock [1], using a prompt structured in OpenAI’s Harmony format. The model returns a JSON object with scores and written feedback for each criterion.

The fine-tuned model uses sequential numeric IDs (1, 2, 3) for criteria and levels, matching the format it was trained on. The application, however, uses UUIDs as primary keys in its database. Before sending a grading request, the backend builds a bidirectional mapping between these two ID systems. It assigns deterministic integer IDs to each criterion and level, sends those to the model in the rubric JSON, and then translates the model’s numeric output back to the corresponding UUIDs when storing the results as `GradingResult` and `CriterionScore` records.

The pipeline uses Celery’s chord primitive to coordinate parallel grading. Each essay is assigned its own chain of two tasks: text extraction followed by grading. The chord groups all of these chains together and attaches the `finalize_essay_batch` callback, which only executes after every chain in the group has completed. This callback checks whether all essays were graded successfully and transitions the assignment from GRADING to REVIEW.

Algorithm 3.1 shows how the grading pipeline is coordinated.

Once grading is complete, the assignment enters the review state. The teacher reviews

3.9. Testing and CI/CD

Algorithm 3.1 Essay Grading Pipeline

Input: Essay IDs $E = \{e_1, e_2, \dots, e_n\}$, Assignment ID a

Output: Each essay is graded and the assignment transitions to REVIEW

- 1: Call WAITFORMODEL to confirm the Bedrock endpoint is responsive
 - 2: **for** each essay $e_i \in E$ **in parallel do**
 - 3: $\text{text}_i \leftarrow \text{EXTRACTESSAYTEXT}(e_i)$ /*MarkItDown: PDF, DOCX, or TXT*/
 - 4: GRADEESSAY(e_i, text_i)
 - 5: Build prompt from writing prompt, source text, rubric JSON, and text_i
 - 6: Send prompt to GPT-OSS-20B on AWS Bedrock
 - 7: Parse JSON response into criterion scores and feedback
 - 8: Create GradingResult and CriterionScore records
 - 9: **end for**
 - 10: FINALIZEESSAYBATCH(a) /*Callback after all parallel tasks complete*/
 - 11: Transition assignment a from GRADING to REVIEW
-

each essay individually, viewing the AI-graded scores and feedback alongside the essay text. For each criterion, the teacher can either accept the AI's score or override it with their own level selection and feedback. Once all the essays in an assignment have been reviewed, the assignment is marked as completed, and results can be exported as a CSV file.

The grading pipeline includes a few safeguards to protect it against failures. If the model returns malformed JSON or missing fields, the task will retry up to 3 times. For errors with AWS Bedrock while the model is still loading, the task will retry 5 times with a 1-minute cooldown between attempts. Before starting a batch, a function probes the model every 5 seconds for up to 5 minutes until it is ready. If one essay fails during extraction or grading, it is marked as FAILED with a reason and shown to the teacher. This only affects that one essay, not the rest in the batch. The teacher can retry failed essays from the frontend.

3.9 Testing and CI/CD

The backend has a full set of tests built with pytest [26]. The tests cover each Django app, including authentication, assignments, and rubrics. I added a mixin that provides methods for creating fake users, rubrics, assignments, and essays using the Faker [18] library.

In the tests, I use a pytest fixture to patch the boto3 AWS Bedrock client so that it can't make any external API calls. This prevents tests from being flaky and saves costs by

Chapter 3. Implementation

mocking inference.

Four GitHub Actions workflows run on each commit. They ensure that all tests pass, that the frontend types match the backend types, and that the code passes the Ruff [6] format check. If the format check fails, the workflow corrects the formatting and pushes the fix automatically.

Chapter 4

Conclusion

4.1 Conclusion

The goal of the project was to build a full-stack application that uses a fine-tuned large language model to help teachers grade essays. InstaGrader was developed as a working system that performs trait-based scoring and evaluates essays against individual rubric criteria rather than producing a single holistic score.

The system uses a fine-tuned version of the `gpt-oss-20b` model. The model was trained on the ASAP and ASAP++ datasets [22, 29], which provide 10,686 essays with per-criterion scores across different rubric dimensions. The fine-tuned model generates structured JSON output, including scores and written feedback, for each criterion based on the teacher’s rubric.

The application implements a complete grading workflow. Teachers can create rubrics from scratch or from a template, then create an assignment with a writing prompt and source material, upload their students’ essays in bulk, and receive AI-generated scores for each essay. The grading pipeline runs in the background using Celery workers, so teachers aren’t blocked from using the platform while essays are being processed.

The main design principle behind InstaGrader is that the teacher maintains control at every step of the marking process. No AI-generated score can be assigned to a student without the teacher reviewing and approving it first. Teachers can accept scores as they are, or override them with their own level and feedback. This approach keeps teachers involved

Chapter 4. Conclusion

and ensures that the tool is designed to support teachers and their decision-making, not replace them.

This project shows that fine-tuned large language models can produce useful, actionable feedback for each criterion of student essays. The combination of the structured rubric input, supervised fine-tuning on human-annotated essay scores, and teachers maintaining full control helped create a system that is responsible and safe to use.

4.2 Future Scope

The next step is preparing InstaGrader for deployment to production. The existing setup runs in a development environment with SQLite and localhost. A production deployment would require containerization, migration to PostgreSQL, HTTPS, proper secret and environment variable setup, and a server to host the application.

Other than deployment, I'd like to add more functionality to the project to make it more useful in a classroom setting. I want to add integrations with Learning Management Systems (LMSs) such as Moodle and Google Classroom so teachers can import student rosters and export grades directly from the platform. This would reduce the amount of manual data entry and work required of teachers. Support for scanned, handwritten or printed essays with OCR would also be beneficial. This would digitize hard-copy essays and allow teachers to grade them on their computers with little effort. Teachers would also benefit from the ability to provide course materials as additional context for the model, beyond what is already present in the reference material field. This could improve scoring accuracy for very specific prompts where the LLM may have limited knowledge.

The fine-tuned model itself has room to grow. Training on larger and more diverse datasets would improve generalization across grade levels and writing styles, beyond what ASAP and ASAP++ cover. Since the start of this project, there have been major developments in open-source LLM technology, and it may make sense to fine-tune another model in place of `gpt-oss-20b`, which is becoming outdated. In April 2026, Google released Gemma 4 [20], which performs significantly better on many benchmarks.

On the implementation side, there are two areas that need some work. I'd like to add a paper-like annotation experience on the frontend, with support for inline highlights,

4.3. Reflections

strikethroughs, and margin comments, similar to what’s available in Microsoft Word. This would give teachers a more familiar way to provide feedback directly on the essay text. The frontend also doesn’t have any unit or end-to-end tests. Adding end-to-end tests for the entire grading workflow would give me more confidence that I’m not breaking anything when I make changes.

4.3 Reflections

The final tech stack is quite different from what I proposed at the beginning of the project. I originally proposed using FastAPI on the backend, Material UI on the frontend, ProseMirror for essay annotations, and Unsloth for fine-tuning the large language model. Over the development process, I reconsidered each of these choices. Table 4.1 summarizes the changes and the reasons behind them.

Table 4.1: Technology pivots from proposal to final implementation.

Component	Proposed	Actual	Reason
Model training	Unsloth (local)	AWS SageMaker AI	Personal machine lacked the hardware for fine-tuning
Backend	FastAPI	Django	Built-in auth, ORM, and admin; familiar from co-op
UI library	Material UI	shadcn/ui	Better React 19 compatibility and customization
Deployment	EC2 + HF Endpoints	Docker + Bedrock	Simplified development and demo workflow
Annotations	ProseMirror	Structured review	Complexity outweighed utility for the scope

The first big pivot I made was in the second week. I originally planned to use Unsloth to fine-tune the model on my own computer and host it via Hugging Face’s Inference Endpoints. When I did more research into what it would take to accomplish this and began testing, I found that my machine was not powerful enough to fine-tune a model of that size, and so I had to find a different solution. I ended up going with the AWS SageMaker AI platform. I uploaded my input and output datasets, adjusted the training parameters, and started

Chapter 4. Conclusion

fine-tuning. The platform provisioned a machine large enough for training. The model took approximately 7 hours to train.

The platform I chose for the frontend and backend also changed. I replaced FastAPI with Django because I had used Django in my previous co-op and knew it included a built-in authentication system, an admin interface, and an ORM. That saved me significant development time because I didn't have to build those components from scratch. On the frontend, I was going to use Material UI, but I ended up pivoting to shadcn/ui because it's more compatible with React 19 and more customizable. I knew I needed extensive customization from whatever component library I chose, since the rubric designer would be a fully custom component.

Some parts of the application were more complicated than I expected. The Celery- and Redis-based grading pipeline was essential to the application, but it added significant complexity around error handling, retry logic, and state management. The rubric designer was the most complex frontend component. It required drag-and-drop reordering of nested lists, validation, and the persistence of the rubric's structure.

Appendix A

Installation and Usage Guide

This appendix walks through how to install and use InstaGrader. It assumes a working knowledge of the command line and basic familiarity with Docker.

A.1 Prerequisites

The only software required on your machine is Docker with Docker Compose. You will also need an AWS account with the following configuration:

1. The AWS CLI must be installed and configured on the host machine so that the application can authenticate with AWS services. Run `aws configure` and provide your access key, secret key, and set the default region to `ca-central-1`. See the AWS CLI documentation¹ for details.
2. Model access must be enabled in the AWS Management Console. Navigate to the Amazon Bedrock console in the `ca-central-1` region, open the Model Access page, and request access to the models you intend to use. See the Amazon Bedrock documentation² for instructions.
3. The application uses Amazon Bedrock for AI grading. Rather than fine-tuning a model, you can use one of the models available through Bedrock, such as Anthropic's Claude

¹<https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html>

²<https://docs.aws.amazon.com/bedrock/latest/userguide/model-access.html>

Appendix A. Installation and Usage Guide

Opus or Claude Sonnet. Once you have selected a model, note the model ID, as you will need it during installation.

A.2 Installation

Start by cloning the repository and navigating to the Docker directory:

```
1 git clone https://github.com/nathanfredericks/instagrader.git
2 cd instagrader/docker
```

Copy the example environment file and open it in a text editor:

```
1 cp .env.example .env
```

Inside the `.env` file, you need to fill in a few values. Set `SECRET_KEY` to a random string (this is used by Django for cryptographic signing), and set `POSTGRES_PASSWORD` to a password of your choice for the database. Set `BEDROCK_MODEL_ID` to the ARN of the fine-tuned model on AWS Bedrock. The rest of the defaults should work out of the box for local development.

Once the environment file is configured, build and start the application:

```
1 docker compose up -d
```

This command pulls the required images, builds the backend and frontend containers, and starts five services: a PostgreSQL database, a Valkey cache (used as the message broker for background tasks), the Django API, a Celery worker for processing grading jobs, and the Next.js frontend. The first build may take a few minutes. Once it finishes, the application is available at `http://localhost:3000`.

To check that all services are running:

```
1 docker compose ps
```

To view logs:

```
1 docker compose logs -f
```

To stop the application:

```
1 docker compose down
```

A.3 Usage

After opening the application in a browser, you will be greeted with a login page. Create a new account by clicking the sign-up link and filling in your details.

A.3.1 Creating a Rubric

Before grading essays, you need a rubric. Navigate to the Rubrics page and click the button to create a new one. You can either build a rubric from scratch by adding criteria and score levels, or start from one of the built-in templates and modify it to fit your needs.

A.3.2 Creating an Assignment

Navigate to the Assignments page and create a new assignment. Give it a title, enter the writing prompt that students were given, optionally add any source text that accompanies the prompt, and select the rubric you want to use for grading.

A.3.3 Uploading Essays

Once the assignment is created, upload student essays. The system accepts PDF, DOCX, and TXT files. You can upload files individually or as a ZIP archive containing multiple essays. After uploading, click the button to start grading.

A.3.4 Reviewing Results

The system grades essays in the background using the fine-tuned model. A progress view shows how many essays have been graded in real time. Once grading is complete, you can review each essay individually. For every criterion in the rubric, the system provides a score and written feedback. You can accept the score as is or override it with your own score and feedback.

Appendix A. Installation and Usage Guide

A.3.5 Completing an Assignment

After reviewing all essays, mark the assignment as complete. You can then export the results as a CSV file for your records.

Bibliography

- [1] Amazon Web Services. Amazon Bedrock – build and scale generative AI applications. <https://aws.amazon.com/bedrock/>, 2024. Accessed 2026.
- [2] Amazon Web Services. Amazon SageMaker AI – machine learning for every developer and data scientist. <https://aws.amazon.com/sagemaker/>, 2024. Accessed 2026.
- [3] Amazon Web Services. Fine-tune models in Amazon SageMaker AI. <https://docs.aws.amazon.com/sagemaker/latest/dg/how-it-works-training.html>, 2024. Accessed 2026.
- [4] Anthropic. Claude sonnet 4.5 model card. <https://www.anthropic.com/news/claude-sonnet-4-5>, 2025. Used for generating synthetic per-criterion feedback.
- [5] Anthropic. Claude opus 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, 2026. Referenced as an example of a large language model.
- [6] Astral. Ruff: An extremely fast Python linter and formatter. <https://github.com/astral-sh/ruff>, 2024. Accessed 2026.
- [7] Yigal Attali and Jill Burstein. Automated essay scoring with e-rater® v. 2. *The Journal of Technology, Learning and Assessment*, 4(3), 2006.
- [8] shadcn Berger. shadcn/ui: Beautifully designed components. <https://ui.shadcn.com>, 2024. Accessed 2026.
- [9] Clauderic Bhatt. dnd kit: A lightweight, performant, accessible drag and drop toolkit for React. <https://dndkit.com>, 2024. Accessed 2026.

Bibliography

- [10] Jill Burstein. The e-rater scoring engine: Automated essay scoring with natural language processing. *Automated essay scoring: A cross-disciplinary perspective*, 113121, 2003.
- [11] Jill Burstein, Karen Kukich, Susanne Wolff, Chi Lu, and Martin Chodorow. Computer analysis of essays. In *NCME symposium on automated scoring*, 1998.
- [12] Tom Christie. Django REST Framework. <https://www.django-rest-framework.org>, 2024. Accessed 2026.
- [13] CoGrader. CoGrader: AI-powered essay grading for teachers. <https://www.cograder.com>, 2024. Accessed 2026.
- [14] Semire Dikli. An overview of automated scoring of essays. *The Journal of Technology, Learning and Assessment*, 5(1), 2006.
- [15] Django Software Foundation. Django: The web framework for perfectionists with deadlines. <https://www.djangoproject.com>, 2024. Accessed 2026.
- [16] Drew Driscoll. openapi-typescript: Generate TypeScript types from OpenAPI 3.0 schemas. <https://github.com/openapi-ts/openapi-typescript>, 2024. Accessed 2026.
- [17] EssayGrader. EssayGrader: AI essay grading assistant. <https://www.essaygrader.ai>, 2024. Accessed 2026.
- [18] Daniele Faraglia. Faker: A Python package that generates fake data. <https://github.com/joke2k/faker>, 2024. Accessed 2026.
- [19] Peter W Foltz, Darrell Laham, and Thomas K Landauer. The intelligent essay assessor: Applications to educational technology. *Interactive Multimedia Electronic Journal of Computer-Enhanced Learning*, 1(2):939–944, 1999.
- [20] Google DeepMind. Gemma 4. <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>, April 2026. Accessed 2026.

Bibliography

- [21] Government of Canada. Personal information protection and electronic documents act (PIPEDA). S.C. 2000, c. 5, 2000. URL <https://laws-lois.justice.gc.ca/eng/acts/p-8.6/>.
- [22] Ben Hamner, Jaison Morgan, lynnvande, Mark Shermis, and Tom Vander Ark. The hewlett foundation: Automated essay scoring. <https://kaggle.com/competitions/asap-aes>, 2012. Kaggle.
- [23] John Hattie and Helen Timperley. The power of feedback. *Review of educational research*, 77(1):81–112, 2007.
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuezhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [25] Timotheus Kampik. drf-spectacular: Sane and flexible OpenAPI 3 schema generation for Django REST Framework. <https://github.com/tfranzel/drf-spectacular>, 2024. Accessed 2026.
- [26] Holger Krekel et al. pytest: Simple powerful testing with Python. <https://docs.pytest.org>, 2024. Accessed 2026.
- [27] Thomas K Landauer, Peter W Foltz, and Darrell Laham. An introduction to latent semantic analysis. *Discourse processes*, 25(2-3):259–284, 1998.
- [28] DJ Madigan and LE Kim. Does teacher burnout affect students. *A systematic review of its association with academic achievement and student-reported*, 2020.
- [29] Sandeep Mathias and Pushpak Bhattacharyya. ASAP++: Enriching the ASAP automated essay grading dataset with essay attribute scores. In Nicoletta Calzolari, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Koiti Hasida, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, Stelios Piperidis, and Takenobu Tokunaga, editors, *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki,

Bibliography

- Japan, May 2018. European Language Resources Association (ELRA). URL <https://aclanthology.org/L18-1187/>.
- [30] Sandeep Mathias and Pushpak Bhattacharyya. Can neural networks automatically score essay traits? In *Proceedings of the fifteenth workshop on innovative use of NLP for building educational applications*, pages 85–91, 2020.
- [31] Meta Platforms. React: A JavaScript library for building user interfaces. <https://react.dev>, 2024. Version 19. Accessed 2026.
- [32] Microsoft. MarkItDown: Document text extraction tool. <https://github.com/microsoft/markitdown>, 2024. Accessed 2026.
- [33] Ellis B. Page. The imminence of grading essays by computer. *Phi Delta Kappan*, 47(5): 238–243, 1966.
- [34] Province of Nova Scotia. Freedom of information and protection of privacy act (FOIPOP). S.N.S. 1993, c. 5, 1993. URL <https://www.canlii.org/en/ns/laws/stat/sns-1993-c-5/latest/sns-1993-c-5.html>.
- [35] Redis Ltd. Redis. <https://redis.io>, 2024. Accessed 2026.
- [36] Mark D Shermis and Jill C Burstein. *Automated essay scoring: A cross-disciplinary perspective*. Routledge, 2003.
- [37] Einar M Skaalvik and Sidsel Skaalvik. Teacher burnout: relations between dimensions of burnout, perceived school context, job satisfaction and motivation for teaching. a longitudinal study. *Teachers and Teaching*, 26(7-8):602–616, 2020.
- [38] Ask Solem. Celery: Distributed task queue. <https://docs.celeryq.dev>, 2024. Accessed 2026.
- [39] Jarred Sumner. Bun: A fast all-in-one JavaScript runtime. <https://bun.sh>, 2024. Accessed 2026.
- [40] Vercel. Next.js: The React framework. <https://nextjs.org>, 2024. Accessed 2026.

Bibliography

- [41] Adam Wathan and Jonathan Reinink. Tailwind CSS: A utility-first CSS framework. <https://tailwindcss.com>, 2024. Accessed 2026.